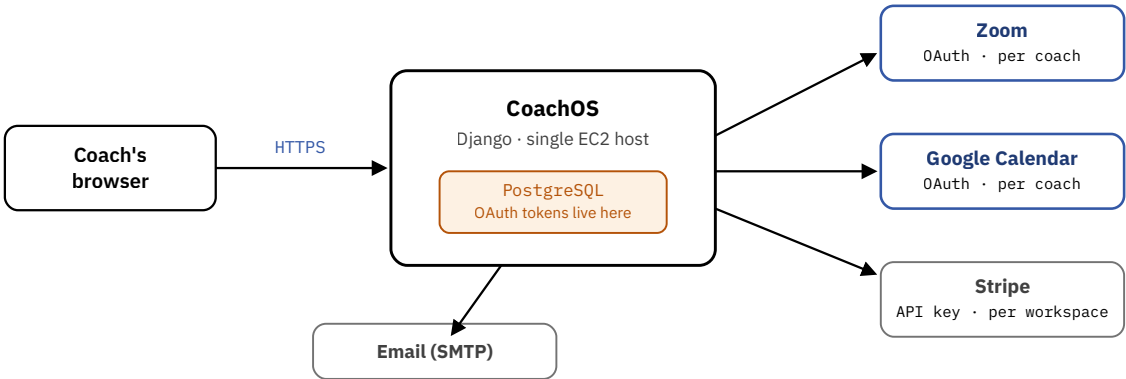


Zoom integration architecture

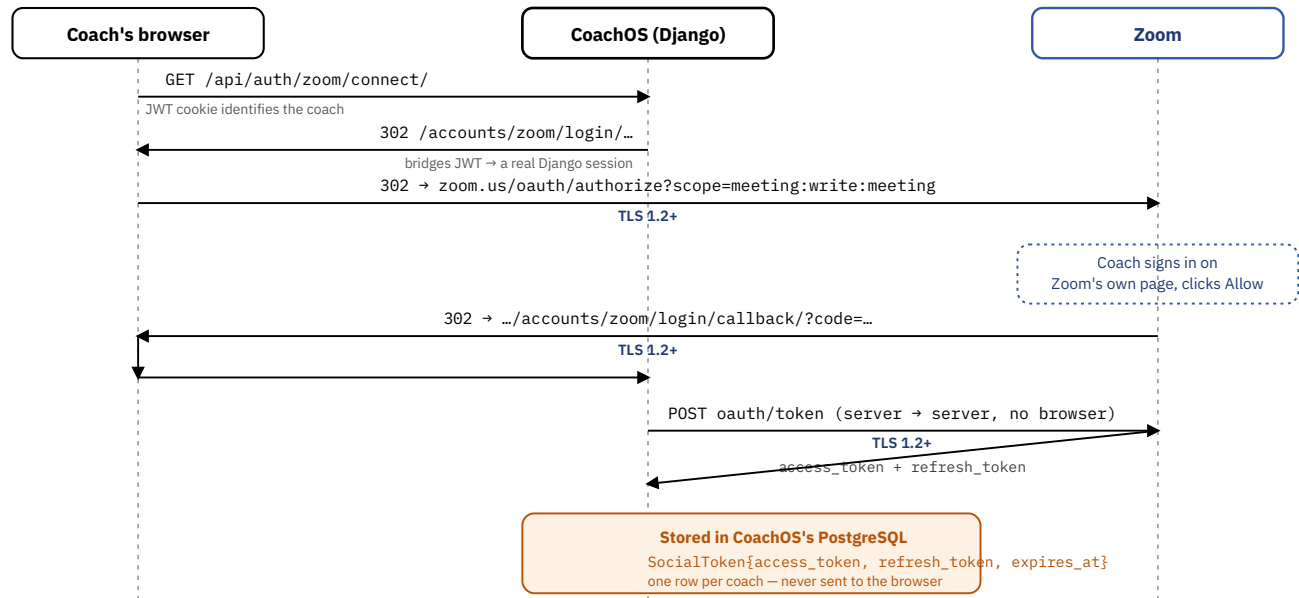
How CoachOS connects a coach's own Zoom account and creates meetings on their behalf — scoped to exactly what this review needs: where OAuth tokens move, where they're stored, and the one API call this integration makes.

WHERE ZOOM SITS IN THE SYSTEM



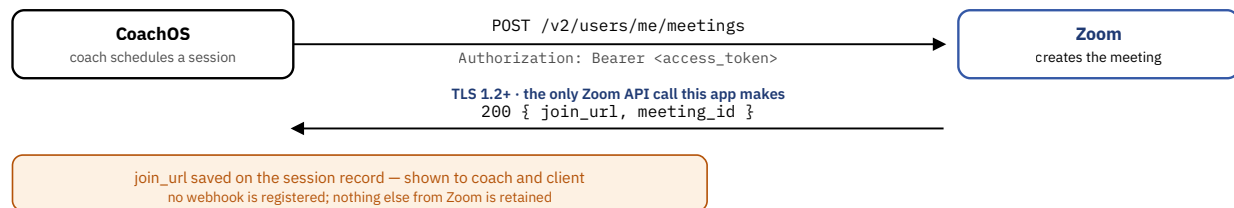
CoachOS is the only system a coach's browser ever talks to. Zoom and Google Calendar are both per-coach OAuth connections — no pooled or shared account; Stripe and email follow different, unrelated patterns shown only for contrast.

CONNECT FLOW — AUTHORIZING A COACH'S ZOOM ACCOUNT



Every hop that leaves a browser or crosses to Zoom runs over TLS 1.2+. The authorization code and tokens never pass through anything but this exchange; the browser only ever sees redirects, never the tokens themselves.

USING THE CONNECTION — CREATING A MEETING



If the stored access token has expired, CoachOS silently refreshes it first using the refresh token (Zoom rotates the refresh token on every use — the new one is saved back) before this call.

SECURITY QUESTIONNAIRE — ANSWERS FROM THE ACTUAL IMPLEMENTATION

Y TLS 1.2+ for all network traffic, including Zoom user's data?

Production CoachOS is HTTPS-only end to end; the one Zoom API call this integration makes (`api.zoom.us`) is TLS-only on Zoom's side regardless.

N Verification tokens / x-zm-signature for incoming Webhook Events?

Not applicable — this integration doesn't subscribe to or receive any Zoom Webhook events at all. Nothing incoming to verify.

Y Does the app collect, store, log, or retain Zoom user data, including OAuth tokens?

Yes — the access token, refresh token, and expiry are stored (one row per connected coach) so the app can create meetings without asking the coach to re-authorize each time. No other Zoom data is retained beyond the created meeting's join link.